

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

R.HARIKA9192572140)

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium) Assessment Tool
Weightage: 35%

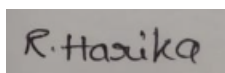
Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5

7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I RASINENI HARIKA(192572140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions,	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs

		functions/modules			
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Complete Query Processing Pipeline

Pseudocode:

QUERY_PROCESS(SQL):

INPUT: SQL query

1. PARSING

tokens $\leftarrow$ LEXICAL_ANALYSIS(SQL)

syntax_tree $\leftarrow$ PARSE(tokens)

2. SEMANTIC ANALYSIS

CHECK_TABLES_COLUMNS(syntax_tree)

CHECK_DATA_TYPES(syntax_tree)

CHECK_AUTHORIZATION(syntax_tree)

3. QUERY REWRITING

relational_algebra $\leftarrow$ CONVERT_TO_RELATIONAL_ALGEBRA(syntax_tree)

optimized_tree $\leftarrow$ APPLY_HEURISTICS(relational_algebra)

// Push selections and projections downward

4. COST-BASED OPTIMIZATION

plans $\leftarrow$ GENERATE_ALTERNATIVE_PLANS(optimized_tree)

best_plan $\leftarrow$ PLAN_WITH_MINIMUM_COST(plans)

5. EXECUTION

result $\leftarrow$ EXECUTE(best_plan)

6. OUTPUT

FORMAT(result)

RETURN result

Pipeline:

SQL Input



Lexical Analysis
↓
Parsing
↓
Semantic Checking
↓
Query Rewriting
↓
Optimization
↓
Execution Plan
↓
Query Execution
↓
Result

2. Cost-Based Query Optimization

The optimizer generates different join orders and chooses the plan with minimum estimated I/O cost.

COST_OPTIMIZER(Relations):

best_plan $\leftarrow$ NULL
minimum_cost $\leftarrow$ INFINITY

plans $\leftarrow$ GENERATE_JOIN_ORDERINGS(Relations)

FOR each plan IN plans:

current_plan $\leftarrow$ plan
total_cost $\leftarrow$ 0

FOR each join IN current_plan:

R $\leftarrow$ join.left_relation
S $\leftarrow$ join.right_relation

cost_NL $\leftarrow$ COST_NESTED_LOOP(R, S)
cost_HJ $\leftarrow$ COST_HASH_JOIN(R, S)
cost_SM $\leftarrow$ COST_SORT_MERGE(R, S)

join_cost $\leftarrow$ MIN(cost_NL, cost_HJ, cost_SM)

total_cost $\leftarrow$ total_cost + join_cost

IF total_cost < minimum_cost:

```
minimum_cost ← total_cost  
best_plan ← current_plan
```

```
RETURN best_plan, minimum_cost
```

For block nested-loop join:

where $B(R)$ and $B(S)$ are page counts and M is the number of buffer pages.

Result: The join ordering with the smallest estimated I/O cost is selected.

3. Nested Loop Join

Nested Loop Join compares each tuple of one relation with tuples of another relation.

NESTED_LOOP_JOIN(R, S):

```
result ← EMPTY
```

```
FOR each tuple r IN R:
```

```
  FOR each tuple s IN S:
```

```
    IF JOIN_CONDITION( $r, s$ ) = TRUE:
```

```
      ADD ( $r, s$ ) TO result
```

```
RETURN result
```

For $|R| = n$ tuples and $|S| = m$ tuples:

For **block nested-loop join**, if:

- $B(R)$ = pages of R
- $B(S)$ = pages of S
- M = available buffer pages

then I/O cost is approximately:

Advantage: Simple and works for any join condition.

Disadvantage: Expensive when both relations are large.

4. Deadlock Detection Using Wait-For Graph

A **Wait-For Graph (WFG)** contains one node for each transaction.

An edge:

$T_i \rightarrow T_j$

means T_i is waiting for a resource held by T_j .

DETECT_DEADLOCK(WFG):

```
visited ← EMPTY
```

```
recursion_stack ← EMPTY
```

```
FOR each transaction T IN WFG:
```

```
  IF T NOT IN visited:
```

```
IF DFS(T, visited, recursion_stack) = TRUE:  
    RETURN "DEADLOCK DETECTED"
```

```
RETURN "NO DEADLOCK"
```

DFS(T, visited, stack):

```
visited.ADD(T)  
stack.ADD(T)
```

FOR each transaction U adjacent to T:

```
IF U NOT IN visited:  
    IF DFS(U, visited, stack) = TRUE:  
        RETURN TRUE
```

```
ELSE IF U IN stack:  
    RETURN TRUE    // Cycle found
```

```
stack.REMOVE(T)
```

```
RETURN FALSE
```

Example:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

A cycle exists, so the system has a **deadlock**.

5. Basic Two-Phase Locking (2PL)

2PL has two phases:

- **Growing phase:** Acquire locks; no lock release.
- **Shrinking phase:** Release locks; no new locks.

BASIC_2PL(Transaction T):

```
phase ← GROWING
```

FOR each operation OP of T:

```
IF OP requires lock:
```

```
IF phase = SHRINKING:  
    ABORT T
```

```
ELSE:  
    IF LOCK_AVAILABLE(OP.data):  
        GRANT_LOCK(T, OP.data)
```

```
ELSE:
  ADD T TO WAIT_QUEUE(OP.data)
  WAIT(T)
```

IF OP is UNLOCK(data):

```
  RELEASE_LOCK(T, data)
  phase ← SHRINKING
```

COMMIT T

RELEASE_ALL_LOCKS(T)

Example:

```
T1:
LOCK-X(A)
LOCK-X(B)
READ/WRITE A,B
UNLOCK(A)
UNLOCK(B)
COMMIT
```

Once T1 releases its first lock, it cannot acquire any new lock.

Result: Basic 2PL guarantees conflict serializability but may cause deadlocks.

6. Timestamp Ordering with Thomas' Write Rule

Each transaction receives a timestamp.

For every data item X:

- Read_TS(X) = largest timestamp of a transaction that read X.
- Write_TS(X) = largest timestamp of a transaction that wrote X.

WRITE(T, X):

```
IF TS(T) < Read_TS(X):
  IGNORE WRITE
  // Thomas' Write Rule
```

```
ELSE IF TS(T) < Write_TS(X):
  IGNORE WRITE
  // Obsolete write
```

```
ELSE:
  WRITE X
  Write_TS(X) ← TS(T)
```

READ(T, X):

```
IF TS(T) < Write_TS(X):  
    ABORT T
```

```
ELSE:  
    READ X  
    Read_TS(X) ← MAX(Read_TS(X), TS(T))
```

Thomas' Write Rule: If a write is already obsolete because a newer transaction has written the item, the obsolete write can simply be **ignored instead of aborting the transaction**.

Benefit: Allows more schedules than basic timestamp ordering while maintaining serializability.

7. ARIES Recovery Algorithm

ARIES performs recovery in three main phases:

1. **Analysis**
2. **Redo**
3. **Undo**

ARIES_RECOVERY(log):

```
// PHASE 1: ANALYSIS  
transaction_table ← EMPTY  
dirty_page_table ← EMPTY
```

```
scan log FROM last_checkpoint
```

```
FOR each log_record:
```

```
    UPDATE transaction_table  
    UPDATE dirty_page_table
```

```
identify:
```

```
    committed_transactions  
    loser_transactions
```

```
// PHASE 2: REDO  
start ← smallest recovery LSN
```

```
FOR each log_record FROM start:
```

```
    IF REDO_REQUIRED(log_record):  
        REDO(log_record)
```

```
// PHASE 3: UNDO  
FOR each loser_transaction T:
```

WHILE T has undoable log records:

record ← LAST_UNDO_RECORD(T)

UNDO(record)

write compensation_log_record(record)

RETURN "RECOVERY COMPLETE"

Analysis: Determines transaction and dirty-page information.

Redo: Repeats necessary updates to restore database state.

Undo: Removes effects of incomplete transactions.

ARIES uses **Write-Ahead Logging (WAL)** and compensation log records during undo.

8. Shadow Paging Recovery

Shadow paging maintains two page tables:

- **Shadow page table:** Stable version.
- **Current page table:** Modified version.

SHADOW_PAGING():

shadow_table ← COPY(current_table)

WHILE transaction is active:

FOR each page update:

new_page ← COPY(old_page)

MODIFY(new_page)

current_table[page] ← new_page

IF COMMIT:

WRITE current_table to stable storage

root_pointer ← current_table

DISCARD shadow_table

IF SYSTEM_FAILURE:

current_table ← shadow_table

root_pointer ← shadow_table

RETURN database

During normal execution:

Shadow Table → old stable pages

Current Table → updated pages

If a crash occurs before commit, the system discards the current page table and uses the shadow page table.

Advantage: No UNDO/REDO log is required.

9. Immediate Update – UNDO/REDO Recovery

In Immediate Update, modified database pages may reach disk before commit. Therefore both **UNDO** and **REDO** may be required.

Example log format:

<T1, X, old_value, new_value>

Pseudocode:

IMMEDIATE_UPDATE_RECOVERY(log):

committed ← EMPTY

uncommitted ← EMPTY

// Find transaction status

FOR each record IN log:

IF record = COMMIT(T):

ADD T TO committed

ELSE IF record = START(T):

ADD T TO uncommitted

// REDO committed transactions

FOR each record IN log:

IF record belongs to committed transaction:

WRITE new_value

// UNDO uncommitted transactions

FOR each record IN log in REVERSE ORDER:

IF record belongs to uncommitted transaction:

WRITE old_value

FLUSH database

RETURN "RECOVERY COMPLETE"

Rule:

Committed transactions → REDO

Uncommitted transactions → UNDO

Using the log's old and new values, the database is restored to a consistent state.

10. Database Connectivity Manager

A connection pool maintains reusable database connections. This avoids creating a new connection for every query.

DATABASE_MANAGER:

```
pool ← CREATE_CONNECTION_POOL()
```

```
GET_CONNECTION():
```

```
  IF pool has available connection:
```

```
    RETURN connection
```

```
  ELSE:
```

```
    WAIT until connection becomes available
```

```
EXECUTE_QUERY(sql, parameters):
```

```
  connection ← GET_CONNECTION()
```

```
  TRY:
```

```
    BEGIN TRANSACTION
```

```
    statement ← PREPARE(connection, sql)
```

```
    FOR each parameter:
```

```
      SET_PARAMETER(statement, parameter)
```

```
    result ← EXECUTE(statement)
```

```
    COMMIT TRANSACTION
```

```
    RETURN result
```

```
  CATCH DatabaseException e:
```

```
    ROLLBACK TRANSACTION
```

```
    LOG(e)
```

```
    THROW e
```

FINALLY:

CLOSE statement

RETURN connection TO pool

Main features:

1. **Connection pooling** reduces connection creation overhead.
2. **Prepared statements** allow parameterized queries.
3. **Commit** makes successful changes permanent.
4. **Rollback** cancels changes when an exception occurs.
5. **Finally block** ensures connections are returned to the pool.

Thus, the manager provides efficient and reliable database connectivity